I M P L E M E N T A T I O N P L A N

openBIMstudio Production Hardening Plan

From vibe-coded prototype to verifiable platform

Repository: github.com/amdghedy/openBIMstudio

Version basis: v0.74.0 (last commit Oct 31, 2026, 610 commits)

Evidence base: README, MASTER_TOOL_CATALOG.md, docs/engineering/BASELINE.md

Audience: project maintainer and contributors

Date: November 1, 2026

Table of Contents

1. Executive Summary....1
2. Current State Diagnosis and Problem Analysis....1
    2.1 Method and Evidence Base....2
    2.2 Findings Mapped to Failure Modes....2
    2.3 What Done Looks Like....3
3. Goals, Principles and Success Criteria....3
    3.1 Objective and Non-Goals....3
    3.2 Five Engineering Principles....3
    3.3 The Definition-of-Done Ladder, Enforced....3
    3.4 Measurable Success Criteria....4
4. Phase 0: Containment (Weeks 1-2)....4
    4.1 Crash Containment: Error Boundaries and a Crash Log....5
    4.2 Input Validation at Every File-Load Boundary....12
    4.3 Backup-on-Load and Before Destructive Writes....16
    4.4 Security: Fail Closed....19
5. Phase 1: The Correctness Engine (Weeks 3-6)....23
    5.1 Retire the window.__obs* Bridges....23
    5.2 TypeScript Strictification....28
    5.3 Golden-File IFC Round-Trip Pipeline....30
    5.4 Property-Based Tests for the Modeling Kernel....37
    5.5 Playwright E2E Smoke Suite on Three Browser Engines....42
6. Phase 2: Structural Trust (Months 2-4)....47
    6.1 Transactional Persistence: Append-Only Journal and Snapshots....47
    6.2 The Maturity Gate: Tool Catalog as Code....51
    6.3 Performance Budgets in CI....56

6.4 Parser Fuzzing in CI.....58
6.5 Specification Before Code.....60
7. Implementation Roadmap and Tracking.....63
8. Risk Analysis and Mitigation.....63
9. Companion GitHub Issue Backlog.....64
10. Appendix: Starter Code Index.....65

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Executive Summary

This plan defines how openBIMstudio graduates from a rapidly evolved, AI-assisted codebase into a platform that degrades predictably instead of breaking on the first edge case. The diagnosis rests on the repository's own evidence: v0.74.0 of the README, the Master Tool Catalog, and the Engineering Baseline, which already document the decisive facts with unusual honesty. The codebase is 75.6 percent JavaScript, relies on pervasive window.__obs* global service bridges, ships no browser-level tests, starts fail-open without a server token, persists project state as non-transactional JSON, and carries a critical remote-code-execution dependency it does not even use. Each of these is a precision-stated finding in the project's own baseline document, not speculation.

The remedy is organized into three phases, each with executable acceptance criteria and ready-to-adopt starter code. Phase 0, Containment, makes failure local instead of global: per-panel error boundaries and a crash log, schema validation at every file-load boundary, backup-on-load, and a fail-closed security posture. Phase 1, the Correctness Engine, builds the verification layer that the project currently lacks: a typed service registry with an ESLint freeze on new globals, TypeScript strictification, a twenty-fixture golden-file IFC round-trip pipeline that doubles as the per-entity interoperability report the baseline says is missing, property-based invariants over the modeling kernel, and a fifteen-scenario Playwright suite running on Chromium, WebKit, and Firefox. Phase 2, Structural Trust, makes honesty machine-enforced: append-only journal persistence with crash recovery, a CI maturity gate wired to the tool catalog, performance budgets, parser fuzzing, and a spec-before-code process rule.

The plan ships with two companion artifacts: a paste-ready GitHub issue backlog (ISSUES.md) and a GitHub CLI script (create_issues.sh) that creates all eighteen issues with labels in one run. The intended outcome is concrete: after Phase 0 the app survives malformed input and crashing tools; after Phase 1 regressions in geometry, undo, or interoperability fail in CI before users see them; after Phase 2 the platform's claims and its test evidence are the same document. The guiding rule throughout: a feature is not done when it works, it is done when breakage attempts fail gracefully.

2. Current State Diagnosis and Problem Analysis

2.1 Method and Evidence Base

The diagnosis below was compiled in November 2026 from four primary sources inside the repository: the README at v0.74.0, documentation/MASTER_TOOL_CATALOG.md (the per-tool maturity catalogue with its status vocabulary of source present, UI exposed,

tested, runtime verified, partial, and not verified), docs/engineering/BASELINE.md (verified format boundaries, feature maturity inventory, known limitations, and risks), and the tests/ directory listing (including priority1-gaps.test.cjs and v40-modules.test.cjs, which explicitly track Revit parity gaps). No claim in this chapter is invented; every finding quotes or paraphrases the project's own documents. This is deliberately a review of what the project already admits, because those admissions are the correct starting point for a hardening roadmap.

2.2 Findings Mapped to Failure Modes

Five clusters of findings explain why the platform breaks on its first edge case. First, weak internal contracts: the repository snapshot reports 75.6 percent JavaScript against 24.1 percent TypeScript, and the baseline flags global architecture issues from pervasive window.__obs* bridges. Any module can mutate shared state through the window object, so an error raised in one tool surfaces as corruption in a different tool, far from its cause. Second, a verification gap: the baseline states plainly that browser-level tests do not exist and that Safari and Firefox are untested, while the tool catalog warns that typecheck is not a runtime test and that a smoke test is not full interoperability validation. Fifty-plus suites verify that modules import and run; they do not verify that BIM behavior is correct.

Third, fail-open behavior: the baseline risk list records fail-open startup when no OBS_SERVER_TOKEN is configured, acceptance of query-string tokens on all /api/ routes, absence of rate limiting, and a phantom dependency on jsonpath-plus (versions at or below 10.2.0) carrying critical remote-code-execution advisories. Fourth, fragile persistence: JSON persistence is described as not transactional with no concurrency control, meaning a crash mid-save or two concurrent editors can produce a corrupt project file with no recovery path. Fifth, interoperability and modeling depth boundaries: authored IFC export is limited to a defined IFC4/IFC2x3 subset with unsupported classes, no per-entity interoperability report exists, BCF 2.1 round-trip validation between two applications is still pending, IDS facets are not itemized, doors and windows host only on walls authored in the app, rooms are non-associative, and the constraint solver has documented solve failures.

These five clusters compound each other, which is the real problem. An unvalidated file (no boundary schema) enters through a parser with no fuzz coverage, mutates state through a global bridge (no typed contract), is persisted non-transactionally (no recovery), in an app with no error boundary (no containment), and no test anywhere notices (no browser-level verification). Vibe-coded platforms are not fragile because the code is bad; they are fragile because nothing bounds the blast radius of the first unexpected input. The table below assigns each finding a severity and points to the phase that removes it.

Table 1: Findings, evidence, severity, and remediation mapping

Finding	Evidence (project documentation)	Severity	Addressed in
Type coverage	75.6 percent JavaScript / 24.1 percent TypeScript (README repository snapshot)	High	Phase 1, Section 5.2
Global architecture	window.__obs* bridges pervasive (BASELINE, global architecture issues)	High	Phase 1, Section 5.1
Browser verification	Browser-level tests: none exist; Safari and Firefox untested (BASELINE)	High	Phase 1, Section 5.5
Security posture	Fail-open startup without OBS_SERVER_TOKEN; query-string tokens on /api/; no rate limiting; jsonpath-plus critical RCE phantom dependency (BASELINE risks)	Critical	Phase 0, Section 4.4
Persistence	JSON persistence not transactional; no concurrency control (BASELINE, collaboration risks)	High	Phase 2, Section 6.1
Interop fidelity	Authored IFC limited subset; no per-entity interop report; BCF round-trip two-application validation pending (BASELINE, Tool Catalog)	Medium	Phase 1 Section 5.3; Phase 2 Section 6.2
Modeling depth	Doors and windows host only on authored walls; rooms non-associative; constraint solver documented failures (Tool Catalog)	Medium	Tracked in backlog; feature work outside this plan

2.3 What Done Looks Like

The success state is not a rewrite. It is the same fast-moving codebase with four properties added: inputs are rejected at the boundary with readable errors; any component failure is contained to its panel and recorded; every behavioral claim in the tool catalog is enforced by a machine; and the model kernel's invariants hold under randomized operation sequences. Everything in this plan serves one of those four properties, and every workstream ends with an acceptance criterion that can be checked in continuous integration rather than in a document.

3. Goals, Principles and Success Criteria

3.1 Objective and Non-Goals

The objective is to make openBIMstudio fail gracefully and verify continuously, while preserving the project's velocity and its MIT, local-first, zero-install character. The plan

deliberately excludes three things: a full rewrite to TypeScript in one pass (too risky for a single maintainer), feature parity work against Revit (important, but the subject of the existing v40-modules tracking, not of hardening), and any change to the product's licensing or deployment model. Hardening here means the difference between a demo and a platform, not a change of direction.

3.2 Five Engineering Principles

Principle one, fail local, never global. A crashing tool must lose its panel, not the user's model; a malformed file must be rejected at the boundary, not corrupt shared state. Principle two, typed boundaries. Every crossing point into the application, whether a file, an API request, a plugin call, or a cross-module service lookup, is validated against an explicit schema or interface. Principle three, transactions. State changes are journaled so that recovery is replay, not repair. Principle four, the verification ladder. Evidence is layered: source code, a visible command, an automated test, and runtime verification with representative models are different levels, and the project already has the vocabulary for this in its tool catalog. Principle five, specification before code. Given AI-assisted development, the bottleneck is not writing code but defining correct; every new tool gets a one-page spec with acceptance criteria and a fixture matrix before implementation, and the agent writes the tests with the feature.

3.3 The Definition-of-Done Ladder, Enforced

The Master Tool Catalog already defines a status vocabulary; Phase 2 turns that vocabulary into a machine-enforced gate. Until then, the ladder below is the reference for every pull request. The essential move is that a status claim without evidence fails the build, which converts the catalog from an honesty document into a contract.

Table 2: Definition-of-Done ladder with CI enforcement

Status	Meaning	CI enforcement (from Phase 2)
Source present	A related source module exists	None (not promotable on its own)
UI exposed	Registered in ribbon, panel, or command palette	Command id must exist in tool-catalog.json
Tested	An automated test covers the behavior	Every listed test file must exist and pass in CI
Runtime verified	Exercised in the browser with representative models	Verified-scenario fixtures must pass golden-file and E2E checks
Partial / known limitation	Documented restriction exists	Limitation text must be non-empty and linked from the UI

3.4 Measurable Success Criteria

The plan is complete when the following key performance indicators are met. They are chosen so that each is a single command in CI, not a judgment call, and the baseline column records the situation documented in the repository today so that progress is unambiguous.

Table 3: Key performance indicators

Metric	Baseline (documented)	Target	Verified by
Browser-level E2E scenarios	None exist (BASELINE)	15 scenarios on Chromium, WebKit, Firefox	Playwright CI job
Golden IFC round-trip fixtures	No interop report (BASELINE)	20 fixtures, 100 percent round-trip pass	golden-roundtrip.yml
Property-based kernel invariants	Not present	10 or more invariants, 500 runs each	fast-check suite
Explicit any in src/modules	Unknown (majority JS)	Zero, ratcheted weekly	ESLint count in CI
Fail-open API endpoints	Token via query string accepted (BASELINE)	Zero; tokens header-only with allow-list	Security integration test
Crash containment	No error boundaries	Every ribbon panel wrapped; crash log viewer shipped	E2E fault-injection scenario
Persistence durability	Not transactional (BASELINE)	Journal replay restores identical state	Recovery property test

4. Phase 0: Containment (Weeks 1-2)

Phase 0 buys safety with the smallest possible diff. None of the four workstreams below rewrites architecture; each installs a boundary at a place where the baseline already documents an unbounded failure. Every item is small enough for one focused day of work and is independently shippable.

4.1 Crash Containment: Error Boundaries and a Crash Log

The single highest-leverage change in this plan is psychological as much as technical: one broken tool must never take down the whole application. React error boundaries achieve this per panel, and a small ring-buffer crash log turns every unhandled error into a diagnosable event instead of a mystery. The log lives in localStorage so it survives reloads, is capped so it can never grow unbounded, and is deliberately defensive against its own corruption. Listing

1 shows the crash log module; Listing 2 shows the boundary component and the one-time global handler installation in the app entry point.

Listing 1: src/core/crashLog.ts - capped ring-buffer crash log

```
export type CrashEntry = {
  ts: string;
  source: 'window.onerror' | 'unhandledrejection' | 'boundary';
  message: string;
  stack?: string;
  tool?: string;
};
const KEY = 'obs.crashlog.v1';
const MAX = 50;

export function recordCrash(entry: Omit<CrashEntry, 'ts'>): void {
  const all = readAll();
  all.unshift({ ...entry, ts: new Date().toISOString() });
  try { localStorage.setItem(KEY, JSON.stringify(all.slice(0, MAX))); }
  catch { /* storage full: dropping old crashes is acceptable */ }
}

export function readAll(): CrashEntry[] {
  try { return JSON.parse(localStorage.getItem(KEY) ?? '[]') as CrashEntry[]; }
  catch { return []; } // a corrupt crash log must never itself crash the app
}

export function exportCrashReport(): string {
  return JSON.stringify(readAll(), null, 2);
}
```

Listing 2: src/shell/AppErrorBoundary.tsx and one-time global handlers

```
import { Component, type ReactNode } from 'react';
import { recordCrash } from '../core/crashLog';

export class AppErrorBoundary extends Component<
  { children: ReactNode; tool: string }, { failed: boolean }> {
  state = { failed: false };
  static getDerivedStateFromError() { return { failed: true }; }
  componentDidCatch(error: Error) {
    recordCrash({ source: 'boundary', message: error.message,
      stack: error.stack, tool: this.props.tool });
  }
  render() {
    if (this.state.failed) return (
      <div className='tool-failure'>This panel failed. The rest of the app is
safe.
        <button onClick={() => this.setState({ failed: false })}>Retry</button>
      </div>);
    return this.props.children;
  }
```

```
}

// In main.tsx, installed once before any feature module loads:
window.addEventListener('error', (e) =>
  recordCrash({ source: 'window.onerror', message: e.message }));
window.addEventListener('unhandledrejection', (e) =>
  recordCrash({ source: 'unhandledrejection', message: String(e.reason) }));
// Wrap every ribbon panel: <AppErrorBoundary tool='clash'> ...
</AppErrorBoundary>
```

4.2 Input Validation at Every File-Load Boundary

The second boundary is the file. Every JSON-shaped input crossing into the application (project files, saved views, BCF manifests, plugin manifests, automation-rule packs) passes through one loader that parses and validates before anything touches application state, returning a typed result instead of throwing. This converts the most common class of vibe-coding crash, the unexpected file, into a readable dialog. Listing 3 shows the loader and three representative schemas. Binary formats such as IFC and DWG get equivalent guards inside their parsers, which Phase 1 fuzzing will verify.

Listing 3: src/core/safeLoad.ts - one validation gate for all JSON-shaped inputs

```
import { z } from 'zod';

export type LoadResult<T> = { ok: true; value: T } | { ok: false; error:
LoadError };
export type LoadError = { stage: 'parse' | 'schema'; file: string; issues:
string[] };

export function safeLoad<T>(file: string, raw: string,
  schema: z.ZodType<T>): LoadResult<T> {
  let json: unknown;
  try { json = JSON.parse(raw); }
  catch (e) { return { ok: false, error: { stage: 'parse', file, issues:
[String(e)] } }; }
  const parsed = schema.safeParse(json);
  if (!parsed.success) return { ok: false, error: { stage: 'schema', file,
    issues: parsed.error.issues.map((i) => i.path.join('.') + ': ' + i.message)
} };
  return { ok: true, value: parsed.data };
}

export const ProjectFileSchema = z.object({
  formatVersion: z.number().int().min(1),
  elements: z.array(z.object({ id: z.string(), type: z.string(),
    props: z.record(z.unknown()) })),
});

export const BcfManifestSchema = z.object({
```

```
  bcf_version: z.literal('2.1'),
  topics: z.array(z.object({ guid: z.string(),
    title: z.string().min(1) })).default([]),
});

export const PluginManifestSchema = z.object({
  id: z.string().regex(/^[a-z0-9-]+$/),
  version: z.string(),
  capabilities: z.array(z.enum(['read-model', 'write-model', 'export',
'network'])),
});
```

4.3 Backup-on-Load and Before Destructive Writes

Because the baseline records persistence as non-transactional, the cheap ninety-percent fix is a rotating backup taken before any load that migrates a file and before any destructive write. If a migration corrupts a project, the user loses nothing. The browser build applies the same policy against IndexedDB or the Origin Private File System; the collaboration server applies it against the filesystem. Listing 4 shows the reference implementation with a keep-last-five rotation.

Listing 4: src/core/backup.ts - timestamped rotating backups (server / Node form)

```
import { promises as fs } from 'fs';
const KEEP = 5;

export async function backupBeforeWrite(path: string): Promise<string> {
  const stamp = new Date().toISOString().replace(/[:.]/g, '-');
  const target = path + '.' + stamp + '.bak';
  try { await fs.copyFile(path, target); }
  catch { /* first ever save: nothing to back up yet */ }
  await pruneOldBackups(path);
  return target;
}

async function pruneOldBackups(path: string) {
  const slash = path.lastIndexOf('/');
  const dir = path.slice(0, slash), base = path.slice(slash + 1);
  const names = (await fs.readdir(dir))
    .filter((f) => f.startsWith(base) && f.endsWith('.bak')).sort();
  for (const old of names.slice(0, Math.max(0, names.length - KEEP)))
    await fs.unlink(dir + '/' + old);
}
```

4.4 Security: Fail Closed

The baseline's security risks are specific and cheap to close. The optional collaboration and conversion server must refuse to start when exposed beyond loopback without a configured token; origins are restricted to an explicit allow-list; tokens travel only in the Authorization

header, never the query string, because URLs end up in logs, history, and Referer headers; and a per-IP token bucket bounds request rates. Separately, the phantom jsonpath-plus dependency is removed and an npm audit gate is added so a critical advisory fails CI rather than shipping silently. Listing 5 shows the middleware; the token bucket is left as a stub with its contract documented.

Listing 5: server/middleware/failClosed.ts - origin allow-list, header-only tokens, rate limit

```
import type { Request, Response, NextFunction } from 'express';

const ALLOWED_ORIGINS = new Set(
  (process.env.OBS_ALLOWED_ORIGINS ?? 'http://localhost:3000').split(','));

export function failClosed(req: Request, res: Response, next: NextFunction) {
  const origin = req.headers.origin ?? '';
  if (origin && !ALLOWED_ORIGINS.has(origin))
    return res.status(403).json({ error: 'origin not allowed' });

  if (typeof req.query.token === 'string')  // tokens must travel in headers
only
    return res.status(401).json({ error: 'query-string tokens are not accepted'
});

  const expected = 'Bearer ' + (process.env.OBS_SERVER_TOKEN ?? '');
  if ((req.headers.authorization ?? '') !== expected)
    return res.status(401).json({ error: 'missing or invalid token' });

  if (!tokenBucket(req.ip ?? 'unknown').tryTake())
    return res.status(429).json({ error: 'rate limited' });
  next();
}

// tokenBucket(ip): fixed-window counter per IP in a Map, 60 requests per
minute.
// Server boot rule: if HOST is not loopback and OBS_SERVER_TOKEN is unset,
exit(1).
```

Table 4: Phase 0 acceptance criteria

Workstream	Acceptance criteria
Crash containment	Every ribbon panel is wrapped in AppErrorBoundary; a fault-injection E2E scenario forces a panel exception and asserts the shell, viewport, and other panels survive; crash log survives reload and is exportable
Input validation	All JSON-shaped loaders route through safeLoad; a corpus of ten malformed inputs (truncated, empty, wrong version, deeply nested, oversized) produces typed LoadError results and zero uncaught exceptions
Backup-on-load	Backups are created before migration and destructive writes; rotation keeps the last five; a test corrupts a project mid-save and verifies recovery from the newest backup

Workstream	Acceptance criteria
Security fail-closed	Server refuses non-loopback start without token; cross-origin requests outside the allow-list receive 403; query-string token receives 401; overload receives 429; npm audit gate blocks critical advisories; jsonpath-plus removed and verified absent from the lockfile

5. Phase 1: The Correctness Engine (Weeks 3-6)

Phase 1 builds the verification layer that turns edge cases from user-reported crashes into red CI checks. It has five workstreams: freezing and retiring the global bridges, strictifying the type system, the golden-file IFC round-trip pipeline, property-based kernel invariants, and the browser-level E2E suite. The order matters: the ESLint freeze (5.1) stops the debt from growing the day it merges, and everything else compounds.

5.1 Retire the window.__obs* Bridges

The baseline names the pervasive __obs* bridges as the project's global architecture issue, and they are the mechanical reason errors travel far from their cause. The fix is incremental: first freeze, then route, then delete. Freeze means an ESLint rule rejects any new window.__obs* access the moment it is written. Route means new code reaches cross-module services exclusively through a typed registry (Listing 7), and existing call sites migrate service by service. Delete means each migrated bridge is removed in the same sprint. The registry throws on duplicate initialization and on use before boot, converting silent wiring bugs into immediate, located failures.

Listing 6: .eslintrc excerpt - freeze on new __obs globals and explicit any*

```
{
  "root": true,
  "parser": "@typescript-eslint/parser",
  "plugins": ["@typescript-eslint"],
  "rules": {
    "no-restricted-syntax": ["error",
      { "selector":
"MemberExpression[object.name='window'][property.name=/^__obs/]",
        "message": "window.__obs* bridges are frozen. Use
src/core/registry.ts." },
      { "selector": "Identifier[name=/^__obs/]",
        "message": "Global __obs* state is frozen. Use the typed registry." }],
    "@typescript-eslint/no-explicit-any": "error"
  },
  "overrides": [
    { "files": ["src/modules/**/*.ts"],
      "rules": { "@typescript-eslint/no-explicit-any": "error" } }
  ]
```

```
}
```

Listing 7: src/core/registry.ts - the typed service registry

```
// The only sanctioned way to reach cross-module services.
export interface BimServices {
  transactions: TransactionService;
  selection: SelectionService;
  ifc: IfcService;
  // migrate one service at a time; keep the interface additive
}

let services: BimServices | null = null;

export function registerServices(s: BimServices): void {
  if (services) throw new Error('registry already initialized: duplicate
boot');
  services = s;
}

export function svc(): BimServices {
  if (!services) throw new Error('registry used before boot: wiring bug');
  return services;
}
```

5.2 TypeScript Strictification

With three quarters of the code in JavaScript, edge cases are invisible at compile time by construction. The migration is deliberately boring and continuous: enable checkJs over the BIM-critical directories first so JavaScript files are type-checked before conversion, then convert files on touch rather than in a big bang, and require all new code to be strict-clean from day one. The weekly any-count is printed in CI and only allowed to fall, a ratchet that makes drift impossible. Geometry, IFC, and transaction modules come first because bugs there are the most expensive; UI feature directories follow.

Listing 8: tsconfig.strict.json - strict baseline for the BIM-critical core

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "exactOptionalPropertyTypes": true,
    "allowJs": true,
    "checkJs": true
  },
  "include": ["src/modules/**/*", "src/core/**/*"]
}
```

5.3 Golden-File IFC Round-Trip Pipeline

This is the single most valuable test a BIM application can own, and it directly closes two documented gaps at once: the missing per-entity interoperability report and the absence of end-to-end format verification. The pipeline imports each fixture IFC, re-exports it through the studio exporter, re-imports the result, and diffs the two models at four levels: per-IFC-class entity counts, property-set key sets, placement matrices, and a normalized tessellation geometry hash. The first run records snapshot baselines; every later run fails CI on any regression and publishes the diff as a JSON artifact. That artifact is the interop report the baseline says does not exist. The recommended starting corpus of twenty fixtures is described in Table 5; all models must be self-authored or openly licensed so they can ship in the repository.

Table 5: Recommended golden-file corpus composition (20 fixtures)

Source exporter	Schema	Discipline	Count	Purpose
Revit, dedicated test project	IFC4	Architecture	4	Primary parity target: walls, doors, roofs, rooms
Revit	IFC2x3	Architecture	2	Legacy schema path
Revit	IFC4	MEP	3	Ducts, pipes, cable trays with fittings
Revit	IFC4	Structure	2	Beams, footings, reinforcement
ArchiCAD	IFC4	Architecture	2	Cross-vendor geometry dialect
Tekla	IFC2x3	Structure	2	Heavy steel assemblies
BlenderBIM (IfcOpenShell)	IFC4	Architecture	2	Open-source exporter baseline
FreeCAD BIM workbench	IFC4	Architecture	1	Open-source exporter baseline
Synthetic minimal	IFC4	Mixed	1	Deterministic unit and placement checks
Synthetic minimal	IFC2x3	Mixed	1	Fuzz seed and unit-tolerance calibration

Listing 9: scripts/golden-roundtrip.mjs - import, re-export, re-import, diff

```
#!/usr/bin/env node
// import -> export -> re-import -> diff; writes interop-report.json; exit 1 on
regression
import { IfcAPI } from 'web-ifc';
import { readdirSync, readFileSync, writeFileSync } from 'fs';
import { diffModel } from '../src/core/interop/diff.js';
import { exportToIfc } from '../src/modules/ifc/export.js';

const CORPUS = 'test/fixtures/ifc-corpus';
const ifc = new IfcAPI();
```

```
await ifc.Init();

let failures = 0;
const report = [];
for (const name of readdirSync(CORPUS).filter((f) => f.endsWith('.ifc'))) {
  const bytes = readFileSync(CORPUS + '/' + name);
  const m1 = ifc.OpenModel(new Uint8Array(bytes), {});
  const reimported = ifc.OpenModel(toBytes(exportToIfc(ifc, m1)), {});
  const entry = { file: name, ...diffModel(ifc, m1, reimported) };
  // diffModel compares per-class entity counts, property-set key sets,
  // placement matrices (1e-6), unit assignments, and a normalized geometry
hash.
  entry.pass = entry.classes.every((c) => c.missing === 0 && c.extra === 0)
            && entry.geomHashEqual && entry.unitsEqual;
  if (!entry.pass) failures++;
  report.push(entry);
  ifc.CloseModel(m1); ifc.CloseModel(reimported);
}
writeFileSync('interop-report.json', JSON.stringify(report, null, 2));
console.log('golden round-trip: ' + (report.length - failures) + '/' +
report.length + ' pass');
process.exit(failures ? 1 : 0);
```

Listing 10: .github/workflows/golden-roundtrip.yml - CI gate and report artifact

```
name: golden-roundtrip
on: [push, pull_request, workflow_dispatch]
jobs:
  roundtrip:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run build
      - run: node scripts/golden-roundtrip.mjs
      - if: always()
        uses: actions/upload-artifact@v4
        with:
          name: interop-report
          path: interop-report.json
      - name: fail on snapshot regression
        run: |
          git diff --exit-code test/fixtures/ifc-corpus/snapshots ||
            (echo '::error::Interop regression: inspect interop-report.json' &&
exit 1)
```

5.4 Property-Based Tests for the Modeling Kernel

Hand-written tests cover the edge cases the author imagined; property-based tests search for the ones nobody imagined, which is exactly the failure mode being fixed. Two invariants matter most to start. The undo invariant: any sequence of modeling operations followed by the same number of undos must restore a state hash identical to the start, which exercises the transaction history the catalog singles out. The rejection invariant: any byte sequence, however malformed, fed to the IFC importer must resolve within a timeout to either success or a typed error, never a hang, an uncontrolled throw, or memory corruption. Ten such invariants across snapping, justification, split and join conservation, and parsing constitute the Phase 1 target.

Listing 11: tests/property/undo-invariant.test.ts - undo restores exact state

```
import fc from 'fast-check';
import { applyOp, undo, hashState, seedModel } from '../helpers/testEngine';

const dx = fc.double({ min: -1000, max: 1000 });
const ops = fc.oneof(
  fc.record({ kind: fc.constant('move'), dx }),
  fc.record({ kind: fc.constant('split') }),
  fc.record({ kind: fc.constant('join') }),
);

fc.assert(
  fc.property(fc.array(ops, { maxLength: 40 }), (seq) => {
    const model = seedModel();
    const before = hashState(model);
    for (const op of seq) applyOp(model, op);
    for (let i = 0; i < seq.length; i++) undo(model);
    return hashState(model) === before;
  }),
  { numRuns: 500 },
);
```

Listing 12: tests/property/ifc-rejection.test.ts - malformed input is controlled failure

```
import fc from 'fast-check';
import { importIfc } from '../../src/modules/ifc/import';

fc.assert(
  fc.asyncProperty(
    fc.uint8Array({ minLength: 0, maxLength: 4096 }),
    async (bytes) => {
      const result = await importIfc(bytes);
      if (result.ok) return true;
      return typeof result.error.code === 'string';  // typed, reportable error
    },
  ),
  { numRuns: 300, timeout: 2000 },
);
```

5.5 Playwright E2E Smoke Suite on Three Browser Engines

The baseline records that browser-level tests do not exist and that Safari and Firefox are untested. A fifteen-scenario Playwright suite, run against the production build on Chromium, WebKit, and Firefox, closes both statements at once and becomes the executable definition of the phrase runs in your browser. WebGL is exercised with the SwiftShader software renderer in CI so the three.js viewport runs headless. The scenario set in Table 6 is deliberately end-to-end: each scenario touches a real user journey, and the final fault-injection scenario depends on the Phase 0 error boundaries.

Table 6: E2E smoke scenarios (run on Chromium, WebKit, Firefox)

ID	Scenario	Asserts
E1	Open app first run	Command center opens; ribbon renders; no console errors
E2	Import wall-demo IFC	Model tree lists IfcWall; viewport canvas has pixels; element count matches fixture
E3	Navigate with ViewCube and presets	Camera state changes; Home restores
E4	Select, hide, isolate	Visibility state changes and panel reflects it
E5	Section box cut	Section plane toggles; no console errors
E6	Create wall, place door	New elements appear in model tree with properties
E7	Move, undo, redo	State hash after undo equals before move; redo re-applies
E8	Save project, reload	Element count and selected properties identical after reload
E9	Run clash detection on fixture pair	Clash list non-empty and status tracking works
E10	Export BCF and re-import	Topic count preserved; titles identical
E11	Create sheet, place view, export PDF	PDF file downloads and is non-empty with expected page count
E12	Schedule quantity to CSV	CSV row count matches visible schedule
E13	Run model validation	Validation report opens and lists known fixture issues
E14	Malformed project file load	Typed error dialog shown; app remains responsive
E15	Fault injection via debug flag	Forced panel exception shows contained failure UI; shell survives; crash log records entry

Listing 13: playwright.config.ts - three engines against the production build

```
import { defineConfig, devices } from '@playwright/test';
```

```
export default defineConfig({
  testDir: 'tests/e2e',
  timeout: 60000,
  retries: process.env.CI ? 1 : 0,
  webServer: {
    command: 'npm run build && npm run preview',
    port: 4173,
    reuseExistingServer: !process.env.CI,
  },
  use: { baseURL: 'http://localhost:4173', video: 'retain-on-failure' },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'],
      launchOptions: { args: ['--use-gl=swiftshader'] } } },
    { name: 'webkit',   use: { ...devices['Desktop Safari'] } },
    { name: 'firefox',  use: { ...devices['Desktop Firefox'] } },
  ],
});
```

Listing 14: tests/e2e/edit-undo-save.spec.ts - the core reliability journey

```
import { test, expect } from '@playwright/test';

test('import, edit, undo, save, reload: state is identical', async ({ page })
=> {
  await page.goto('/');
  await page.getByTestId('import-ifc')
    .setInputFiles('tests/fixtures/ifc-corpus/wall-demo.ifc');
  await expect(page.getByTestId('model-tree')).toContainText('IfcWall');
  const countBefore = await page.getByTestId('element-count').textContent();

  await page.getByTestId('tool-move').click();
  await page.mouse.click(400, 300);
  await page.mouse.click(500, 320);
  await page.keyboard.press('Control+z');

  await page.getByTestId('save-project').click();
  await page.reload();
  await expect(page.getByTestId('element-count')).toHaveText(countBefore ??
'');
});
```

Table 7: Phase 1 acceptance criteria

Workstream	Acceptance criteria
Global freeze	ESLint rule blocks window.__obs* and bare __obs* identifiers in CI; registry boots all services used by the shell; at least three services migrated out of bridges
TypeScript strict	tsconfig.strict.json passes over src/modules and src/core with allowJs/checkJs; weekly any-count ratchet wired into CI and non-increasing
Golden round-trip	Twenty-fixture corpus committed with licenses; interop-report.json generated on every push; snapshot regression fails CI; report downloadable as artifact

Workstream	Acceptance criteria
Property invariants	Ten or more invariants green at 500 runs each, including undo and rejection; a seeded failure injected into the undo engine is caught by the property test
E2E suite	Fifteen scenarios pass on all three engines against the production build; flaky scenarios quarantined with an issue rather than skipped silently

6. Phase 2: Structural Trust (Months 2-4)

Phase 2 changes what the project is, not just what it does: state becomes recoverable, claims become machine-enforced, and regressions in speed or robustness become build failures. The five workstreams below are larger than Phase 0 items and are sequenced so each lands value independently.

6.1 Transactional Persistence: Append-Only Journal and Snapshots

The baseline records JSON persistence as not transactional and collaboration as lacking concurrency control. The design fix is the same pattern used by every durable system: an append-only operation journal plus periodic state snapshots. Every user action appends one line; every five hundred operations the full state is snapshotted and the journal compacts. Recovery is deterministic: load the newest snapshot, replay the operations after it, and arrive at an identical state, a property that is itself tested. The journal also becomes the substrate for collaboration: remote peers receive the same operation stream, which makes concurrency control a question of sequencing operations rather than reconciling divergent JSON blobs. Listing 15 sketches the core class.

Listing 15: src/core/journal.ts - append-only journal with snapshot compaction

```
export type Op = { seq: number; ts: number; kind: string; payload: unknown };

export class Journal {
  private constructor(private store: JournalStore, private nextSeq: number) {}

  static async open(store: JournalStore): Promise<Journal> {
    const snap = await store.loadNewestSnapshot();
    const j = new Journal(store, snap ? snap.seq + 1 : 1);
    await j.replayAfter(snap);   // deterministic: same ops, same state
    return j;
  }

  async append(kind: string, payload: unknown): Promise<Op> {
    const op: Op = { seq: this.nextSeq++, ts: Date.now(), kind, payload };
    await this.store.appendLine(JSON.stringify(op));  // durable before this
returns
    if (op.seq % 500 === 0) await this.snapshot();
```

```
    return op;
  }

  private async snapshot(): Promise<void> {
    const seq = this.nextSeq - 1;
    await this.store.writeSnapshot(seq, serializeState());
    await this.store.compactJournalBefore(seq);
  }
}
```

6.2 The Maturity Gate: Tool Catalog as Code

The Master Tool Catalog is the project's most valuable honesty artifact, but a markdown table cannot fail a build. The Phase 2 move is to encode the catalog as documentation/tool-catalog.json (Listing 16) and to enforce it with a script (Listing 17) that runs on every push. The script extracts every command id registered in the UI and fails if any command is missing from the catalog; it then fails if any tool claiming the tested or runtime-verified status does not have all its declared test files present and passing. The markdown catalog remains as the human-readable rendering, generated from the JSON so the two can never drift. This is the mechanism that turns the project's own vocabulary of source present, UI exposed, tested, and runtime verified from documentation into contract.

Listing 16: documentation/tool-catalog.json - the catalog as data

```
{
  "tools": [
    { "id": "clash.detect", "name": "Clash Detection", "status": "tested",
      "tests": ["tests/clash-detection.test.cjs"], "verifiedScenarios": [] },
    { "id": "wall.create", "name": "Wall Tool", "status": "runtime-verified",
      "tests": ["tests/wall.test.cjs", "tests/e2e/wall.spec.ts"],
      "verifiedScenarios": ["fixtures/ifc-corpus/wall-demo.ifc"] },
    { "id": "rooms.edit", "name": "Rooms", "status": "partial",
      "knownLimitation": "non-associative; plan panel unwired" }
  ]
}
```

Listing 17: scripts/check-tool-catalog.mjs - the enforcement gate

```
#!/usr/bin/env node
// Every UI command must be catalogued; every status claim must have real test
files.
import { readFileSync, existsSync, readdirSync } from 'fs';

const catalog = JSON.parse(readFileSync('documentation/tool-catalog.json',
'utf8'));
const commandIds = scanCommandRegistry('src/commands');  // parse
registerCommand ids
const errors = [];

for (const id of commandIds)
```

```
  if (!catalog.tools.some((t) => t.id === id))
    errors.push('UI command not in catalog: ' + id);

for (const tool of catalog.tools) {
  if (['tested', 'runtime-verified'].includes(tool.status))
    for (const t of tool.tests ?? [])
      if (!existsSync(t))
        errors.push(tool.id + ' claims ' + tool.status + ' but missing test: '
+ t);
  if (tool.status === 'partial' && !tool.knownLimitation)
    errors.push(tool.id + ' is partial but has no knownLimitation text');
}

if (errors.length) { console.error(errors.join('\n')); process.exit(1); }
console.log('tool catalog consistent: ' + commandIds.length + ' commands
checked');
```

6.3 Performance Budgets in CI

Browser applications regress silently: each release adds features and the large-model experience decays until a user complains. A budget makes the regression a red cross instead. A reference large IFC (fifty to one hundred megabytes, publicly describable, committed with a checksum rather than the blob) is imported in a headless benchmark that records wall-clock time and peak memory; a budget file declares the ceilings; CI fails when either is exceeded. Bundle size for the main script gets the same treatment. Budgets start generous, are tightened only after three consecutive green runs, and are never loosened without an issue explaining why.

Listing 18: scripts/budget.json and the benchmark invocation

```
// scripts/budget.json
{
  "import":   { "fixture": "bench/large-reference.ifc.checksum",
                "maxSeconds": 60, "maxPeakMB": 1500 },
  "bundle":   { "maxMainJsKB": 2500 },
  "e2eSmoke": { "maxSeconds": 180 }
}

// CI invocation:
//   node --expose-gc scripts/bench-large-model.mjs   -> prints
{ importSeconds, peakHeapMB }
//   scripts/check-budget.mjs compares the output to budget.json and exits 1 on
breach.
// Rule: tighten only after three consecutive green runs; loosening requires an
issue.
```

6.4 Parser Fuzzing in CI

web-ifc and LibreDWG are native WebAssembly modules: malformed input does not merely throw a JavaScript error, it can crash the tab or exhaust memory. The fuzzing loop mutates bytes of the golden corpus with deterministic seeds, runs each mutation in a disposable worker with a hard timeout, and accepts exactly two outcomes: success or a typed error. A hang, an out-of-memory kill, or a worker crash is a release blocker. Running nightly at five hundred iterations per parser (IFC, BCF, DXF) keeps the cost negligible while continuously re-proving the controlled-failure property that Phase 1 established with property tests.

Listing 19: scripts/fuzz-ifc.mjs - mutation loop with controlled-failure gate

```
#!/usr/bin/env node
// Mutate corpus bytes; the parser may fail, but only in a controlled way.
import { readFileSync } from 'fs';
import { importIfcIsolated } from '../src/modules/ifc/importWorker';

const seeds = ['test/fixtures/ifc-corpus/wall-demo.ifc',
               'test/fixtures/ifc-corpus/synth-minimal.ifc'];

for (let i = 0; i < 500; i++) {
  const bytes = Buffer.from(readFileSync(seeds[i % seeds.length]));
  for (let k = 0; k < 20; k++) bytes[(i * 31 + k * 7) % bytes.length] ^= 1 <<
(k % 8);
  const outcome = await runWithTimeout(() => importIfcIsolated(bytes), 10000);
  if (outcome !== 'ok' && outcome !== 'typed-error') {
    console.error('uncontrolled failure at iteration ' + i + ': ' + outcome);
    process.exit(1);   // hang, OOM kill, or worker crash = release blocker
  }
}
console.log('fuzz: 500 mutations, all outcomes controlled');
```

6.5 Specification Before Code

The last workstream is procedural and it is the one that makes the rest permanent. The failure mode of AI-assisted development is not that code is written badly, it is that nobody paused to define what correct means before the code existed. The countermeasure is a one-page tool spec, shown in Listing 20, that every feature, whether human- or agent-written, must have before implementation starts. The spec names the user job, the acceptance criteria as numbered verifiable statements, the fixture matrix that proves it, the failure modes it must survive, and the catalog status it must reach to ship. The agent then writes the tests from the spec alongside the feature, and CI referees. Over time the spec repository becomes the project's true requirements document, written at the speed of development.

Listing 20: documentation/SPEC_TEMPLATE.md - one page per tool, before code

```
# Tool Spec: <tool.id>

```

```
Problem: which user job this serves; which documented Revit-parity gap it
closes.
Scope: explicitly in scope; explicitly out of scope.

Acceptance criteria (each independently verifiable):
1. ...
2. ...
3. ...

Fixture matrix: which corpus models prove it (schemas, disciplines, edge
cases).
Failure modes that must be handled: malformed input, empty input, oversized
input,
  undo mid-operation, offline mode, concurrent edit.
Catalog target: the status this tool must reach before shipping (tested or
above),
  and the test files that will prove it.
```

Table 8: Phase 2 acceptance criteria

Workstream	Acceptance criteria
Journal persistence	Every state mutation is journaled; a kill-during-save test recovers to an identical state hash via snapshot plus replay; journal compaction keeps files bounded; collaboration server reads the same operation stream
Maturity gate	tool-catalog.json exists and renders the markdown catalog; check-tool-catalog.mjs runs on push; an intentionally uncatalogued command added in a test branch fails the build
Performance budgets	Benchmark runs in CI on every release candidate; budget breach fails the build; budgets tightened once after three green runs to prove the mechanism
Parser fuzzing	Nightly fuzz runs 500 mutations per parser (IFC, BCF, DXF); an injected hang in a test branch fails the run; no uncontrolled outcomes tolerated
Spec-before-code	SPEC_TEMPLATE.md merged; every new tool PR links a spec; a PR without a spec is closed by process, not by exception

7. Implementation Roadmap and Tracking

The roadmap assumes a single maintainer working part-time on hardening alongside feature development, which is why every workstream is shippable alone and nothing requires a coordinated big-bang merge. The windows are indicative; the exit criteria are the contract. The issue identifiers reference the companion backlog delivered with this plan.

Table 9: Roadmap and exit criteria

Phase	Window	Exit criteria	Issue ids
Phase 0	Weeks 1-2	Error boundaries on all panels; safeLoad on all JSON loaders; backup-on-load active; server fail-closed; jsonpath-plus removed	P0-01 to P0-05
Phase 1	Weeks 3-6	Global freeze linted; strict config green on core; 20-fixture round-trip gating CI; 10 property invariants; 15 E2E scenarios on 3 engines	P1-01 to P1-07
Phase 2	Months 2-4	Journal persistence live; catalog gate enforcing; budgets and nightly fuzz gating; spec process adopted for all new tools	P2-01 to P2-06

Tracking uses the repository's own project board with four columns: Backlog (issues), Spec (issues with a linked one-page spec), In review (pull request with tests), and Verified (merged and enforced in CI). The weekly cadence for a solo maintainer has three rituals. First, the any-count ratchet review: confirm the count did not rise. Second, the catalog review: promote or demote at most a handful of tool statuses, with evidence. Third, the flake review: any test that failed intermittently is either fixed or quarantined with an issue the same week, never left red and ignored.

Three process rules keep the codebase honest without slowing it down. Rule one, failing test first: every bug found in the wild becomes a failing test before the fix, so the suite becomes an autobiography of every edge case the platform survived. Rule two, review-sized diffs: even working solo, self-review each diff after twenty-four hours; experience shows this catches roughly a third of defects before merge. Rule three, spec before code: no feature PR without its one-page spec, enforced socially until the maturity gate makes it mechanical.

8. Risk Analysis and Mitigation

The plan itself carries risks, and naming them is part of the honesty standard the project already set. Each risk below has a mitigation that is part of the plan rather than a hope.

Table 10: Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Single-maintainer bandwidth; hardening competes with features	High	Medium	Phase 0 items are one-day diffs; Phase 1 workstreams are independent; the catalog gate keeps scope honest; hardening tasks are sized to fit a feature-adjacent week

Risk	Likelihood	Impact	Mitigation
Round-trip diffs are noisy because authored IFC export is a documented subset	Medium	High	Diff at class level with tiered expectations per fixture (strict for synthetic, subset-aware for vendor files); report missing and extra separately; tighten tiers as the exporter grows
WebGL rendering in CI is slow or flaky on software rasterizers	Medium	Medium	SwiftShader flags in the Playwright config; pixel assertions kept minimal (canvas non-blank); heavier visual checks left to local runs
Fixture licensing blocks committing the corpus	Low	High	Corpus is self-authored or openly licensed by policy (Table 5); vendor-exported fixtures use dedicated test projects, and checksums replace blobs where redistribution is unclear
Flaky tests erode trust in the suite	Medium	Medium	One retry in CI only; weekly flake review; quarantine column with a linked issue; a red build always has an owner
Strictification surfaces a wave of pre-existing type errors that blocks work	High	Low	checkJs before conversion reports without blocking; the any-ratchet only ratchets down after the initial count is accepted; UI directories migrate last

One risk is deliberately accepted: this plan does not attempt to fix the documented modeling-depth gaps (hosted openings on imported models, room associativity, constraint solver failures). Those are feature work, they are already tracked by the project's own Revit-gap tests, and mixing them into hardening would blur both. The backlog keeps them visible and out of scope.

9. Companion GitHub Issue Backlog

This plan ships with two files that make execution mechanical. The first, ISSUES.md, contains eighteen fully drafted issues (P0-01 through P2-06) with priority, phase, estimate, evidence quotations from the project's own baseline, task lists, and acceptance criteria, formatted to paste directly into GitHub. The second, create_issues.sh, automates that: it creates the labels and opens every issue through the GitHub CLI in one run, so the backlog exists in the repository within minutes.

To use the script, authenticate once with gh auth login, run bash create_issues.sh from the repository root (or any directory, since issue creation targets origin by name), and review the opened issues. The script is idempotent by title convention: re-running it after editing bodies will open duplicates, so treat it as a one-shot bootstrap and manage issues in GitHub afterwards. Each issue body ends with its acceptance criteria so that closing the issue is a checklist, not a judgment.

Table 11: Backlog summary (details in ISSUES.md)

Id	Title	Priority	Phase
P0-01	Global crash containment: error boundaries per panel + crash log viewer	Critical	Phase 0
P0-02	Zod validation at all file-load boundaries (safeLoad)	Critical	Phase 0
P0-03	Backup-on-load with rotating timestamps	High	Phase 0
P0-04	Fail-closed server: origin allow-list, header-only tokens, rate limiting	Critical	Phase 0
P0-05	Remove jsonpath-plus; add npm audit CI gate	Critical	Phase 0
P1-01	Freeze __obs* globals with ESLint; typed service registry	High	Phase 1
P1-02	TypeScript strictification wave 1 (src/modules, src/core)	High	Phase 1
P1-03	Assemble 20-fixture IFC golden corpus (licensed, snapshotted)	High	Phase 1
P1-04	Golden round-trip pipeline + interop report + CI workflow	High	Phase 1
P1-05	Property-based kernel invariants (fast-check)	High	Phase 1
P1-06	Malformed-input rejection property tests (IFC, BCF, DXF)	Medium	Phase 1
P1-07	Playwright E2E smoke suite on Chromium, WebKit, Firefox	High	Phase 1
P2-01	Append-only journal persistence with snapshot recovery	High	Phase 2
P2-02	Maturity gate: tool-catalog.json enforcement in CI	High	Phase 2
P2-03	Performance budget benchmark + CI gate	Medium	Phase 2
P2-04	Nightly parser fuzzing loop	Medium	Phase 2
P2-05	Spec-before-code process + SPEC_TEMPLATE.md	Medium	Phase 2
P2-06	Surface per-entity IFC interop report in the UI	Medium	Phase 2

Dependencies between issues are recorded inside the issue bodies where they matter: P1-03 and P1-04 are sequential, P1-07 scenario E15 depends on P0-01, and P2-02 should land before any status promotion in the catalog is accepted. The board columns described in Chapter 7 map one-to-one onto these dependencies.

10. Appendix: Starter Code Index

The listings in Chapters 4 through 6 are starter implementations: correct in shape and contract, intended to be adapted to the codebase's real service names and store APIs rather than pasted blindly. The table below indexes each artifact, its target path in the repository, and the phase that owns it, so this document can serve as the working checklist during implementation.

Table 12: Code artifact index

Artifact	Target path in repository	Phase	Listing
Crash log module	src/core/crashLog.ts	Phase 0	Listing 1
Error boundary + global handlers	src/shell/AppErrorBoundary.tsx, main.tsx wiring	Phase 0	Listing 2
Boundary loader + schemas	src/core/safeLoad.ts	Phase 0	Listing 3
Rotating backups	src/core/backup.ts (browser: OPFS equivalent)	Phase 0	Listing 4
Fail-closed middleware	server/middleware/failClosed.ts	Phase 0	Listing 5
ESLint freeze rules	.eslintrc (no-restricted-syntax, no-explicit-any)	Phase 1	Listing 6
Typed service registry	src/core/registry.ts	Phase 1	Listing 7
Strict TypeScript config	tsconfig.strict.json	Phase 1	Listing 8
Golden round-trip pipeline	scripts/golden-roundtrip.mjs + src/core/interop/diff.ts	Phase 1	Listing 9
Round-trip CI workflow	.github/workflows/golden-roundtrip.yml	Phase 1	Listing 10
Undo invariant property test	tests/property/undo-invariant.test.ts	Phase 1	Listing 11
Malformed-input property test	tests/property/ifc-rejection.test.ts	Phase 1	Listing 12
Playwright config	playwright.config.ts	Phase 1	Listing 13
Core E2E spec	tests/e2e/edit-undo-save.spec.ts	Phase 1	Listing 14
Operation journal	src/core/journal.ts	Phase 2	Listing 15
Tool catalog as data	documentation/tool-catalog.json	Phase 2	Listing 16
Catalog enforcement gate	scripts/check-tool-catalog.mjs	Phase 2	Listing 17
Performance budget	scripts/budget.json, scripts/bench-large-model.mjs	Phase 2	Listing 18
Parser fuzz loop	scripts/fuzz-ifc.mjs	Phase 2	Listing 19
Tool spec template	documentation/ SPEC_TEMPLATE.md	Phase 2	Listing 20

The document should be read alongside the three repository files that motivated it: README.md for the feature claims, documentation/MASTER_TOOL_CATALOG.md for the per-tool maturity vocabulary, and docs/engineering/BASELINE.md for the verified limitations and risks that this plan systematically closes. Where this plan and the baseline disagree after any Phase 2 item lands, the baseline is updated first and the plan second; the catalog gate makes forgetting that impossible.